

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

ассистент

должность, уч. степень, звание

подпись, дата

И.Д. Свеженин

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №4

Нативный пользовательский интерфейс;

по курсу: Кроссплатформенное программирование

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков

инициалы, фамилия

Санкт-Петербург 2025

СОДЕРЖАНИЕ

1. Задание	2
2. Ход работы.....	3
3. Результат работы программы.....	4
4. Вывод.....	7
5. Листинг с кодом программы.....	8

1. Задание

Задание данной лабораторной работы заключалось в создании мобильного приложения под операционную систему Android на языке программирования Kotlin. Темой выбранной предметной области стали автомобили. Основной целью являлась разработка приложения для управления списком автомобильных задач, где пользователь может добавлять новые задачи, отмечать их как выполненные и удалять при необходимости. Для отображения списка использовался элемент RecyclerView, обеспечивающий динамическое обновление интерфейса. В процессе выполнения работы требовалось закрепить навыки работы с XML-разметкой, настройкой элементов интерфейса и реализацией логики на Kotlin. Итогом стало приложение — простой менеджер задач, предназначенный для планирования и контроля действий, связанных с автомобилем.

2. Ход работы

Работа началась с создания нового проекта в Android Studio с использованием языка Kotlin и шаблона Empty Activity. В файле `activity_main.xml` была определена основная структура интерфейса: поле ввода текста для новой задачи, кнопка добавления и список задач на основе `RecyclerView`. Для элементов списка использовалась отдельная разметка `item_task.xml`, включающая чекбокс для отметки выполнения задачи, `ImageView` для прикрепленного фото и кнопки для добавления фото и удаления задачи. При проектировании интерфейса были учтены удобство пользователя и простота взаимодействия с приложением.

В файле `MainActivity.kt` была реализована основная логика приложения. Создан класс `Task` для хранения информации о задаче, включая заголовок, статус выполнения и URI прикрепленного изображения. Для отображения списка задач использован адаптер `TaskAdapter`, связывающий данные с элементами `RecyclerView`. В адаптере реализованы обработчики событий для чекбокса, кнопки удаления и кнопки прикрепления фото. Выбор изображения из галереи устройства осуществляется с помощью `ActivityResultContracts.GetContent`, после чего выбранное изображение привязывается к конкретной задаче и отображается в списке.

В процессе работы также была настроена динамическая работа списка: при добавлении задачи вызывается метод `notifyItemInserted`, при удалении — `notifyItemRemoved`, что позволяет интерфейсу мгновенно реагировать на изменения. Пользователь может добавлять новые задачи, отмечать их как выполненные, удалять и прикреплять фотографии, что делает приложение удобным инструментом для планирования и контроля действий, связанных с автомобилем. Итогом выполнения работы стала полностью функционирующая часть приложения, реализующая управление списком задач с поддержкой прикрепления изображений.

3. Результат работы программы

В качестве результата работы программы представлен вывод значений в терминал. На рисунках 1-2 показан пример работы программы с задачами.

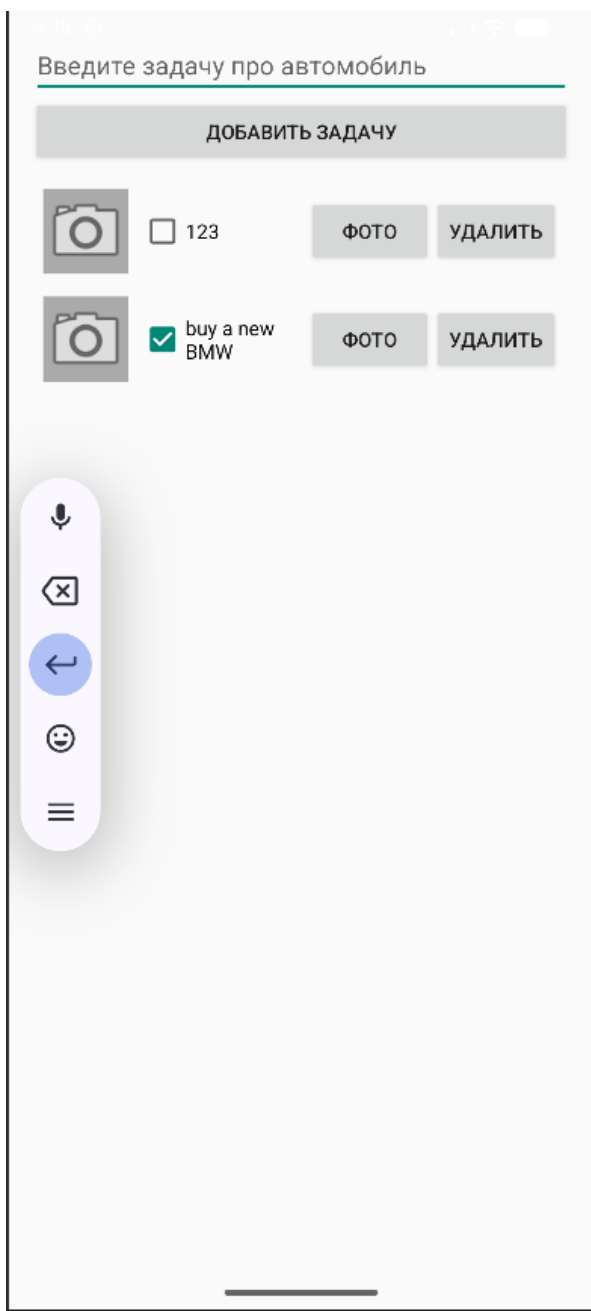


Рисунок 1 – Добавление задачи

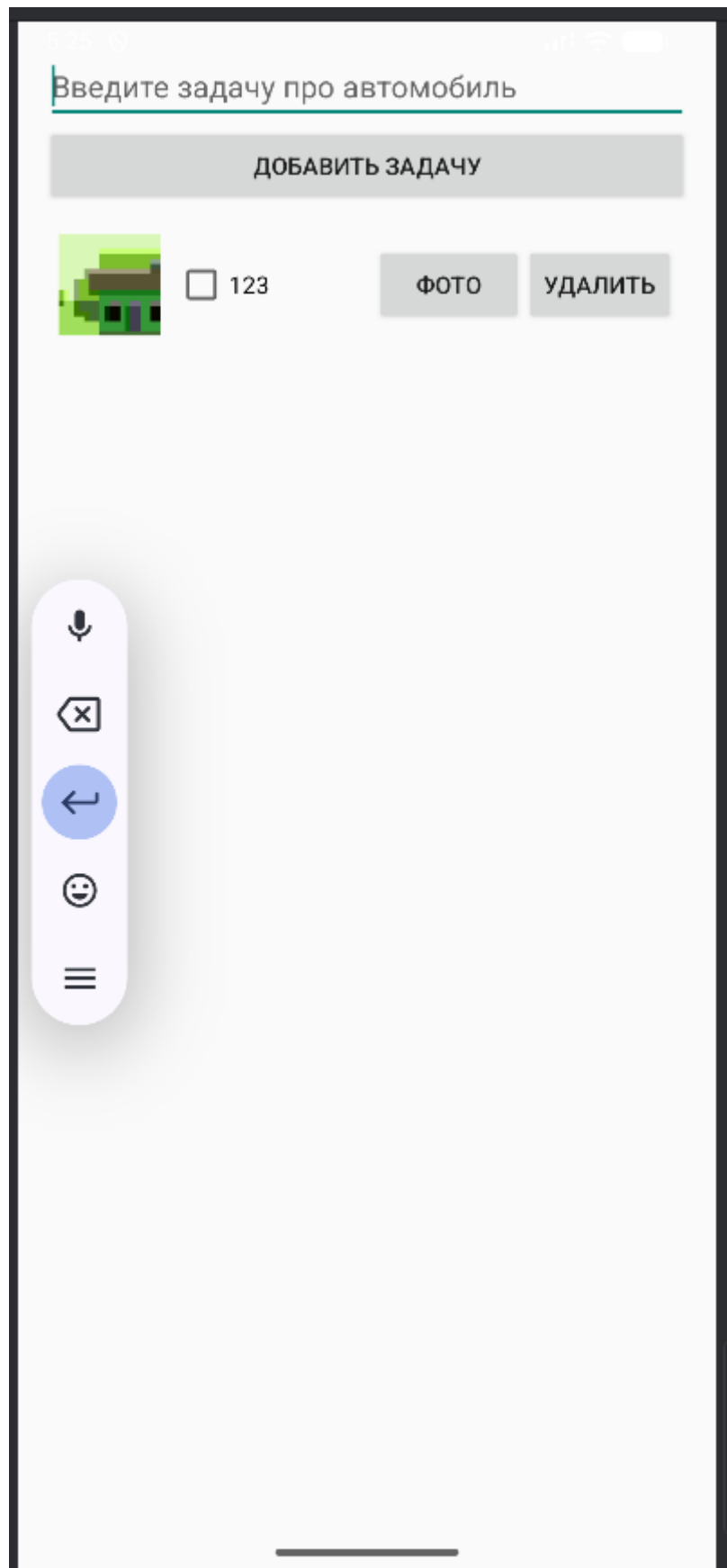


Рисунок 2 – Добавление фото, удаление пункта

4. Вывод

В ходе выполнения лабораторной работы было успешно разработано мобильное приложение для управления списком автомобильных задач. В процессе работы были изучены и применены основы работы с Android Studio, XML-разметкой и языком Kotlin, а также освоены методы взаимодействия визуальных элементов с логикой приложения через адаптеры и обработчики событий.

Была реализована полная функциональность по добавлению, удалению и отметке задач как выполненных, а также возможность прикрепления изображений к отдельным задачам с динамическим обновлением интерфейса через RecyclerView. Такой подход обеспечивает удобство использования приложения и наглядное отображение информации для пользователя.

Итогом работы стало работоспособное мобильное приложение, позволяющее эффективно планировать и контролировать задачи, связанные с автомобилем, с поддержкой визуальных материалов. Реализованная структура проекта и логика приложения позволяют легко расширять функционал, например, добавлением отдельного экрана галереи изображений или других элементов управления.

5. Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:

```
#mainactivity.kt
```

```
package com.example.autotasks
```

```
import android.net.Uri
```

```
import android.os.Bundle
```

```
import android.view.LayoutInflater
```

```
import android.view.View
```

```
import android.view.ViewGroup
```

```
import android.widget.Button
```

```
import android.widget.CheckBox
```

```
import android.widget.EditText
```

```
import android.widget.ImageView
```

```
import androidx.activity.result.contract.ActivityResultContracts
```

```
import androidx.appcompat.app.AppCompatActivity
```

```
import androidx.recyclerview.widget.LinearLayoutManager
```

```
import androidx.recyclerview.widget.RecyclerView
```

```
class MainActivity : AppCompatActivity() {
```

```
    private lateinit var etTask: EditText
```

```
    private lateinit var btnAdd: Button
```

```
    private lateinit var recyclerView: RecyclerView
```

```
    private val taskList = ArrayList<Task>()
```

```
    private lateinit var adapter: TaskAdapter
```

```
    // хранит индекс задачи, к которой прикрепляется фото
```

```
    private var currentAttachIndex: Int = -1
```

```

private val pickImageLauncher =
    registerForActivityResult(ActivityResultContracts.GetContent()) { uri:
Uri? ->

    uri?.let {
        if (currentAttachIndex in taskList.indices) {
            taskList[currentAttachIndex].imageUri = it
            adapter.notifyItemChanged(currentAttachIndex)
        }
    }
}

```

```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    etTask = findViewById(R.id.etTask)
    btnAdd = findViewById(R.id.btnAdd)
    recyclerView = findViewById(R.id.recyclerView)

    adapter = TaskAdapter(taskList)
    recyclerView.layoutManager = LinearLayoutManager(this)
    recyclerView.adapter = adapter

    btnAdd.setOnClickListener {
        val text = etTask.text.toString().trim()
        if (text.isNotEmpty()) {
            taskList.add(Task(text))
            adapter.notifyItemInserted(taskList.size - 1)
            etTask.text.clear()
        }
    }
}

```

```

    }
}
}

```

```

data class Task(var title: String, var done: Boolean = false, var imageUri:
Uri? = null)

```

```

inner class TaskAdapter(private val items: ArrayList<Task>) :
RecyclerView.Adapter<TaskAdapter.TaskViewHolder>() {

```

```

    inner class TaskViewHolder(view: View) :
RecyclerView.ViewHolder(view) {
        val imgPhoto: ImageView = view.findViewById(R.id.imgPhoto)
        val checkBox: CheckBox = view.findViewById(R.id.checkBox)
        val btnAttach: Button = view.findViewById(R.id.btnAttach)
        val btnDelete: Button = view.findViewById(R.id.btnDelete)
    }

```

```

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
TaskViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_task, parent, false)
        return TaskViewHolder(view)
    }

```

```

    override fun onBindViewHolder(holder: TaskViewHolder, position: Int)
{
        val task = items[position]
        holder.checkBox.text = task.title
        holder.checkBox.isChecked = task.done

```

```

        // если фото прикреплено — отображаем
        if (task.imageUri != null) {
            holder.imgPhoto.setImageURI(task.imageUri)
        } else {

holder.imgPhoto.setImageResource(android.R.drawable.ic_menu_camera)
        }

        holder.checkBox.setOnCheckedChangeListener { _, isChecked ->
            task.done = isChecked
        }

        holder.btnAttach.setOnClickListener {
            currentAttachIndex = holder.adapterPosition
            pickImageLauncher.launch("image/*")
        }

        holder.btnDelete.setOnClickListener {
            val pos = holder.adapterPosition
            items.removeAt(pos)
            notifyItemRemoved(pos)
        }
    }

    override fun getItemCount(): Int = items.size
}
}

```

#activity_main.xml

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp"
    tools:context=".MainActivity">

    <EditText
        android:id="@+id/etTask"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Введите задачу про автомобиль" />

    <Button
        android:id="@+id/btnAdd"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Добавить задачу" />

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recyclerView"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:paddingTop="8dp" />

</LinearLayout>

```

#item_image

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="4dp">
```

```
<ImageView
```

```
    android:id="@+id/imageView"
    android:layout_width="match_parent"
    android:layout_height="120dp"
    android:scaleType="centerCrop"
    android:background="@android:color/darker_gray" />
```

```
</FrameLayout>
```

#item_task

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<LinearLayout
```

```
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_vertical"
    android:padding="8dp">
```

```
<ImageView
```

```
    android:id="@+id/imgPhoto"
    android:layout_width="60dp"
```

```

        android:layout_height="60dp"
        android:layout_marginEnd="8dp"
        android:scaleType="centerCrop"
        android:src="@android:drawable/ic_menu_camera"
        android:background="@android:color/darker_gray" />

```

```
<CheckBox
```

```

        android:id="@+id/checkBox"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1" />

```

```
<Button
```

```

        android:id="@+id/btnAttach"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Фото" />

```

```
<Button
```

```

        android:id="@+id/btnDelete"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Удалить" />

```

```
</LinearLayout>
```

```
#styles.xml
```

```
<resources xmlns:tools="http://schemas.android.com/tools">
```

```

    <style
        name="Theme.AutoTasks"

```

```

        parent="Theme.AppCompat.DayNight.NoActionBar">

```

```

        <item name="colorPrimary">@color/black</item>

```

```
<item name="colorPrimaryVariant">@color/black</item>
<item name="colorOnPrimary">@color/white</item>
<item                                name="android:statusBarColor"
tools:targetApi="l">?attr/colorPrimaryVariant</item>
</style>
</resources>
```